



RIFCO

MVP Product and Technical Blueprint Packet

Distribution edition for leadership, product, operations, and development alignment

Document Purpose	Formal build blueprint for the launch and staged execution of RIFCO.
Product Positioning	Launch first as an on demand Oyster Shucker app, then expand into a broader request an experience platform after the data and pricing engine is validated.
Target MVP Window	90 to 120 days
Locked Build Direction	React Native mobile app, Firebase backend, native modules for live tracking and background location, Next.js admin and ops tooling on Vercel.

This blueprint is designed to be used as an execution document, a progress tracker, a communication handoff, and a high level reference for mind mapping and milestone planning.

Table of Contents

1. Executive Summary
2. Product Vision and Launch Strategy
3. Product Principles and Scope Guardrails
4. Locked Technology Stack
5. System Architecture Overview
6. Core Product Modules
7. Algorithm and Intelligence Engine Strategy
8. Pricing Engine Design
9. Matching and Dispatch Logic
10. Data Collection and Event Architecture
11. Firebase Backend Blueprint
12. Mobile App Blueprint
13. Native Module Blueprint
14. Admin and Operations Blueprint
15. 90 to 120 Day MVP Roadmap
16. Phase by Phase Build Tasks
17. Milestones, Gates, and Success Metrics
18. Expansion Path After Oyster Shucking
19. Whiteboard Mind Map Guide
20. Final Build Summary

1. Executive Summary

RIFCO should launch as a focused on demand service application centered on a single wedge category: Oyster Shuckers. This narrow launch is intentional. It simplifies operations, sharpens the marketing message, reduces onboarding complexity, and gives the business a controlled environment for collecting high quality demand, fulfillment, distance, pricing, and timing data.

The long term company is not simply an oyster shucker booking app. It is a request based experience platform powered by a strong intelligence engine that continuously improves pricing, matching, zone logic, provider quality scoring, and future category expansion. The launch category trains the system; it does not define the ceiling of the company.

The recommended MVP stack is: React Native with TypeScript for the mobile app, Firebase for the backend, Swift and Kotlin native modules for live tracking and background location workflows, and a Next.js admin and operations platform on Vercel. This stack provides speed, simultaneous iOS and Android launch capability, and enough flexibility to add more advanced infrastructure later without overengineering the first product.

The central technical principle is simple on the front end, sophisticated underneath. The consumer experience should feel clean and obvious: request service, see pricing, confirm booking, track the provider, complete payment. Underneath, the platform should be collecting structured event data and running continuous pricing, demand, and dispatch logic.

2. Product Vision and Launch Strategy

2.1 Launch Positioning

The public launch version of RIFCO is an app that lets customers request an Oyster Shucker for a party, event, private gathering, or hospitality setting. The launch message should be easy to understand within seconds. The focus should remain narrow during the MVP stage.

2.2 Why the Oyster Shucker Wedge is Strategically Strong

- It creates a clear and differentiated launch story.
- It allows a controlled supply side because the provider type is highly specific.
- It enables cleaner pricing experiments because the service baseline is consistent.
- It reduces product sprawl during the MVP build window.
- It generates category specific data that can later be generalized into a broader request an experience platform.

2.3 Long Term Expansion Vision

Once the pricing engine, dispatch logic, provider quality signals, and operating cadence are validated, RIFCO expands into adjacent categories such as bartenders, chefs, hibachi chefs, magicians, musicians, and other event based service experiences. Category expansion should be staged and deliberate. The company should not add multiple new experiences until the first category is operationally clean and the algorithmic foundation is reliable.

2.4 Product Experience Standard

RIFCO should not feel like a broad marketplace. It should feel like a premium request app. The user should always experience clarity, speed, and confidence. Complexity belongs behind the scenes, not in the interface.

3. Product Principles and Scope Guardrails

Principle	Meaning	MVP Implication
Focused wedge first	Launch with only Oyster Shuckers	Do not add broader categories during MVP build
Simple customer flow	Fast and obvious booking path	Minimize settings, branching paths, and optionality
Rich event data	Every meaningful action becomes a tracked event	Data architecture is a priority, not an afterthought
Rules first, prediction later	Start with clear scoring and pricing formulas	Do not launch with opaque black box models
Operational visibility	Admin tools must support live monitoring	Build ops dashboards early, not after launch
Native only where needed	Use native modules for high sensitivity workflows	Keep most UI and business flows in React Native

4. Locked Technology Stack

Layer	Recommended Choice	Purpose	Reasoning
Mobile app	React Native with TypeScript	Shared iOS and Android application	Fastest clean path to simultaneous cross platform launch
Backend	Firebase	Auth, Firestore, Functions, Storage, notifications, analytics	Strong fit for real time application behavior and fast MVP execution
Native capability layer	Swift and Kotlin modules	Live tracking, background location, map intensive workflows	Native quality where React Native alone is not enough
Web and ops	Next.js on Vercel	Landing page, admin panel, support tooling, ops dashboard	Fast deployment workflow and strong internal tooling surface
Payments	Stripe	Customer payment flow and future payout support	Reliable payment infrastructure and extensibility
Maps and routing	Google Maps Platform	Map rendering, geocoding, ETA and route visualization	Strong support for location based service flows
Presence layer	Firebase Realtime Database if needed	Provider online state and lightweight live heartbeat style status	Useful for presence scenarios that are cleaner outside Firestore

5. System Architecture Overview

The platform architecture should be separated into four major layers.

Architecture Layer	Core Responsibilities	Primary Technology
Shared product layer	Customer UI, provider UI, booking flows, pricing display, status screens, profile and settings	React Native
Native infrastructure layer	Background location, map performance workflows, location update handling, permission edge cases	Swift and Kotlin modules
Backend logic layer	Request lifecycle, pricing calculations, matching triggers, notifications, event logging, analytics aggregation	Firebase Functions and Firestore
Operations layer	Provider approval, live job oversight, pricing controls, cancellation review, reporting	Next.js on Vercel

6. Core Product Modules

6.1 Customer Side Modules

- Account creation and sign in
- Location entry and address selection
- Request creation flow
- Pricing display and confirmation
- Live provider tracking
- Booking status timeline
- Payment and receipt view
- Ratings and review submission
- Profile, history, and support surfaces

6.2 Provider Side Modules

- Provider sign up and profile
- Approval and vetting state
- Go online and go offline controls
- Incoming request flow
- Accept and decline behavior
- Navigation and active trip state
- Job start and job complete actions
- Earnings history and basic performance visibility

6.3 Admin and Operations Modules

- Provider approval queue
- Live requests monitor
- Live jobs monitor
- Cancellation review
- Pricing rule controls
- Zone and surge monitoring
- Customer and provider support actions
- Analytics and KPI summary view

7. Algorithm and Intelligence Engine Strategy

The intelligence engine should not begin as one giant machine learning system. The correct approach is layered: event collection first, decision logic second, learning and optimization third, predictive modeling fourth.

Stage	Engine Type	Main Objective	Build Form
Stage 1	Event collection engine	Capture structured data from every meaningful action	Tracked events, logs, and snapshots
Stage 2	Rules and scoring engine	Compute price, zone pressure, and provider ranking	Cloud Functions and configurable formulas
Stage 3	Optimization engine	Adjust weights based on historical outcomes	Scheduled recalculation jobs and dashboard calibration
Stage 4	Predictive intelligence	Forecast demand, conversion, cancellations, and ideal price bands	Future model training after real usage volume exists

7.1 What the Engine Must Learn Over Time

- Which time windows convert at the highest rates
- Which neighborhoods sustain premium pricing
- How much distance customers tolerate before conversion falls
- Which providers reliably accept and complete high value requests
- What price ranges create strong fulfillment without hurting demand
- When same day or near term urgency should create a premium
- Which categories or adjacent services should be launched next

8. Pricing Engine Design

The pricing engine should be transparent, formula based, and adjustable during the MVP stage. Every pricing output must be logged so that the team can study performance and refine weights.

Input	Description	Why It Matters	Phase 1 Handling
Base service fee	Starting fee for the Oyster Shucker service	Creates a stable economic baseline	Static admin controlled amount
Distance fee	Charge based on provider travel burden	Protects provider economics and affects conversion	Radius or mileage based rule
Time premium	Adjustment for evening, weekend, or prime event windows	Captures natural demand differences	Schedule based rule bands
Demand multiplier	Adjustment based on open requests versus available providers	Handles hot zones and supply pressure	Zone pressure scoring
Urgency premium	Adjustment for short lead time or immediate requests	Reflects operational difficulty	Same day or near term rule
Manual controls	Admin overrides during launch and calibration	Protects service quality during early operations	Dashboard override

8.1 Surge Pricing Logic

Surge pricing should be based on zone level demand pressure, not arbitrary guesses. Each zone should receive a live pressure score. That score should map into a bounded multiplier range so that prices remain explainable and controlled.

Zone Band	Suggested Meaning	Pricing Range
Normal	Balanced demand and supply	1.0x
Busy	Healthy demand increase	1.1x to 1.2x
High Demand	Clear pressure and lower supply	1.3x to 1.5x
Hot Zone	Strong imbalance requiring premium pricing	1.6x to 1.9x
Extreme Demand	Rare situations with heavy pressure	Capped threshold

8.2 Distance Pricing Logic

Distance pricing should begin with a clean and simple model. The system should estimate provider travel burden using radius or mileage based logic. More advanced traffic and route complexity handling can be added later. During the MVP phase, simplicity and clarity are more important than overfitting.

8.3 Time Based Pricing Logic

The system should understand weekday versus weekend demand, day versus evening differences, and same day urgency. These rules should be configurable so the team can calibrate based on real booking patterns.

9. Matching and Dispatch Logic

The matching engine should rank providers using a score rather than a first come first served mechanism. The first version does not need to be overly complex, but it should account for the inputs that most clearly affect quality and fulfillment.

Signal	Meaning	Why It Matters	MVP Weighting Goal
Availability	Provider is online and eligible	Primary requirement	Highest
Distance	Proximity to event location	Affects ETA and economics	High
Reliability	Completion and cancellation behavior	Protects service quality	High
Responsiveness	Acceptance and response time history	Improves booking speed	Medium to high
Quality	Ratings and repeat customer indicators	Protects customer experience	Medium
Category fit	Relevant skill and approved service type	Ensures service suitability	Required gate

Later stages can add predictive acceptance scoring, cancellation risk scoring, and dynamic provider activation logic. The MVP should focus first on reliable fulfillment and understandable ranking behavior.

10. Data Collection and Event Architecture

This is one of the most important parts of the build. The product can be simple, but the data layer cannot be sloppy. Every meaningful action should become a structured event.

10.1 Required Customer Events

- user signed up
- address searched
- request created
- price quoted
- price accepted
- price abandoned
- booking confirmed
- provider tracked
- job completed
- job cancelled
- tip added
- review submitted
- repeat request created

10.2 Required Provider Events

- provider signed up
- provider approved
- provider went online

- provider went offline
- request received
- request accepted
- request declined
- provider en route
- provider arrived
- job started
- job completed
- job cancelled

10.3 Required System Snapshots

- active requests by zone
- available providers by zone
- request to provider ratio by zone
- average quoted price by zone
- match success rate by zone
- completion rate by zone
- cancellation rate by zone
- average ETA by zone and time window

11. Firebase Backend Blueprint

Firebase is the recommended backend because it supports rapid MVP development, real time state updates, event triggered logic, notifications, and a strong mobile centric development workflow.

Firestore Service	Primary Use	Blueprint Notes
Firebase Auth	Customer and provider authentication	Use role aware account structure and approval states
Cloud Firestore	Core application records	Users, providers, requests, jobs, ratings, pricing logs, zone snapshots
Cloud Functions	Business logic and scheduled jobs	Price calculations, request lifecycle triggers, notifications, aggregations
Cloud Storage	Profile photos, docs, and media assets	Use for provider documents and app assets where appropriate
Firebase Cloud Messaging	Push notifications	Booking updates, provider actions, status changes
Realtime Database	Presence and lightweight live state if needed	Useful for online state and heartbeat style availability handling
Firebase Analytics	Usage and conversion tracking	Use with custom event schema for product intelligence

11.1 Core Firestore Collections

- users
- providers
- providerApprovalQueue
- requests
- jobs
- pricingLogs
- zoneSnapshots
- ratings
- supportCases
- systemConfig
- analyticsEvents or equivalent event storage pattern

11.2 Cloud Function Families

- on request creation
- on price quote generation
- on provider acceptance
- on job state update
- on cancellation
- scheduled zone snapshots
- scheduled provider score refresh
- scheduled pricing weight refresh preparation
- notification dispatch functions

12. Mobile App Blueprint

12.1 Shared React Native App Responsibilities

- Customer and provider onboarding interfaces
- Shared navigation architecture
- Request flow screens
- Pricing and quote presentation
- Status and booking lifecycle screens
- Ratings, history, profile, support, and settings
- General API and Firebase integration layer

12.2 Suggested Major Screen Groups

Screen Group	Representative Screens	Priority
Auth and onboarding	Welcome, sign in, sign up, role selection, basic profile setup	High
Customer request flow	Home, location entry, request details, pricing quote, confirmation	High
Live booking flow	Tracking, status timeline, provider details, contact or support surface	High
Provider workflow	Provider home, incoming request, active job, completion, earnings summary	High
Account and support	History, profile, support, settings	Medium
Optional future surfaces	Promo logic, referrals, deeper earnings breakdowns	Low for MVP

13. Native Module Blueprint

React Native should remain the main application shell, but high sensitivity workflows should use native modules for reliability and performance.

Native Module Area	Why Native Support is Recommended	Example Responsibilities
Background location	Platform specific behavior and reliability concerns	Location updates while provider is active and en route
Map intensive workflows	Smooth updates and rendering quality	Marker movement, route display, location smoothing
Permission handling edge cases	Platform specific flows can be cleaner natively	Location permission escalation and background rules
Performance sensitive live tracking	Improved responsiveness for active jobs	Provider movement updates and trip state coordination

14. Admin and Operations Blueprint

An early stage on demand service app requires strong operational visibility. Admin tools should not wait until after launch. They are part of the MVP.

Admin Module	Purpose	Why It Matters
Provider approval panel	Review and approve Oyster Shucker applicants	Controls supply quality
Live request monitor	Watch open customer demand and matching state	Supports launch operations
Live jobs monitor	Track active service fulfillment	Helps with support and intervention
Pricing controls	Adjust base price, premiums, and surge thresholds	Critical during calibration period
Zone dashboard	See demand and supply by area	Supports surge and hot zone management
Cancellation review	Track why jobs fail or are cancelled	Feeds future product and pricing improvements
Support tools	Assist customers and providers quickly	Operational trust layer
Analytics summary	Review KPIs and trends	Supports management decisions

15. 90 to 120 Day MVP Roadmap

Phase	Timeline Window	Primary Goal	Key Deliverables	Gate to Exit Phase
Phase 1	Weeks 1 to 2	Foundation and architecture lock	Scope, stack, data schema, lifecycle states, event taxonomy, pricing logic v1	Blueprint signed off
Phase 2	Weeks 2 to 4	UX and product flow design	High fidelity flows, navigation map, state diagrams, design system basics	UX approved
Phase 3	Weeks 4 to 10	Core mobile app build	Auth, customer and provider flows, request lifecycle UI, tracking surfaces	Primary mobile flows functioning
Phase 4	Weeks 4 to 10 in parallel	Backend and logic layer	Firestore model, Functions, pricing engine v1, notifications, event logging	Live backend logic functioning
Phase 5	Weeks 8 to 11	Admin and ops dashboard	Provider approval, live monitor tools, pricing controls, analytics overview	Ops can monitor launch
Phase 6	Weeks 10 to 14	Testing and calibration	Scenario testing, pricing tuning, cancellation review, internal simulations	Soft launch ready
Phase 7	Weeks 13 to 17	Controlled launch	City launch, provider pool activated, live monitoring and support	Public release active

16. Phase by Phase Build Tasks

16.1 Phase 1, Foundation and System Design

- Confirm launch wedge: Oyster Shucker only
- Lock mobile, backend, payment, maps, and admin stack
- Define customer and provider role architecture
- Define request lifecycle states
- Define provider lifecycle states
- Define pricing inputs and formula skeleton
- Define event taxonomy and snapshot strategy
- Define zones or area segmentation strategy
- Draft admin dashboard functional requirements

16.2 Phase 2, UX and Product Flow Design

- Map complete customer request journey
- Map provider accept and fulfill journey
- Design quote presentation and confirmation flow
- Design active tracking screen
- Design provider online and offline control flow
- Design ratings and history flow
- Design provider approval and admin operations interfaces
- Finalize visual hierarchy and reusable components

16.3 Phase 3, Core Mobile Build

- Implement authentication and role based access
- Build onboarding and account setup
- Build request creation flow
- Build quote display and confirmation flow
- Build active job and tracking flow
- Build provider request handling screens
- Build ratings and review flow
- Integrate push notification handling
- Implement profile, history, and settings

16.4 Phase 4, Backend and Logic Layer

- Create Firestore schema and security rules
- Implement request creation and update functions
- Implement pricing engine v1 in Cloud Functions
- Implement surge score calculation
- Implement provider selection logic
- Implement notification triggers
- Implement event logging and zone snapshot jobs
- Implement admin configuration storage

16.5 Phase 5, Admin and Operations Build

- Build provider approval queue
- Build live request monitor
- Build live jobs monitor
- Build pricing and threshold controls
- Build zone heat view or equivalent zone reporting view
- Build cancellation review panel
- Build analytics summary cards and trend panels

16.6 Phase 6, Testing and Calibration

- Run end to end booking simulations
- Test provider online and offline behavior
- Test location updates and tracking reliability
- Test pricing formulas across scenarios
- Review edge cases and support actions
- Refine thresholds and multipliers
- Validate launch day operational checklist

17. Milestones, Gates, and Success Metrics

Milestone	Definition of Done	Owner Lens	Success Signal
Architecture locked	Stack, schema, states, and roadmap approved	CTO	No major technical ambiguity remains
Flows approved	Customer, provider, and admin UX defined	Product and design	Build can proceed without product confusion
Core booking live	A request can be created, priced, matched, tracked, and completed	Engineering	Internal test cycle works end to end
Pricing engine stable	Rules produce understandable and tunable outputs	Engineering and ops	Calibration possible without code churn
Ops dashboard usable	Team can monitor live activity and make adjustments	Operations	Launch support readiness
Soft launch ready	Critical bugs reduced and support playbook exists	Leadership	Confidence for controlled rollout

17.1 MVP KPI Starter Set

- quote acceptance rate
- booking completion rate
- provider acceptance rate
- provider cancellation rate
- average time to match
- average ETA
- average job value
- repeat request rate
- customer rating average
- provider reliability score distribution

18. Expansion Path After Oyster Shucking

After the Oyster Shucker category proves operationally and economically sound, the business can expand to adjacent service categories. The expansion should be intentional, one category at a time, using the same intelligence framework and fulfillment discipline.

Expansion Step	What Gets Added	Precondition to Launch
Step 1	Bartenders	Stable pricing and fulfillment metrics in Oyster Shucking
Step 2	Chefs or Hibachi Chefs	Operational confidence in multi hour event style jobs
Step 3	Musicians or entertainers	Confidence in category specific pricing and provider vetting
Step 4	Broader request an experience surface	Algorithm and operations mature enough to support diversification

19. Whiteboard Mind Map Guide

The best way to convert this blueprint into a visual mind map is to center the board around the product and then branch outward into the major company systems.

- Center node: RIFCO
- Branch 1: Customer app
- Branch 2: Provider app
- Branch 3: Pricing engine
- Branch 4: Matching and dispatch
- Branch 5: Live tracking and native modules
- Branch 6: Firebase backend
- Branch 7: Admin and ops dashboard
- Branch 8: Event tracking and analytics
- Branch 9: Launch wedge, Oyster Shuckers
- Branch 10: Future category expansion

Under Pricing Engine, map inputs such as base fee, distance fee, time premium, demand multiplier, urgency premium, and admin override. Under Firebase Backend, map Auth, Firestore, Functions, Storage, Messaging, Analytics, and optional Realtime Database. Under the Admin and Ops branch, map provider approval, live monitors, pricing controls, cancellation review, and KPI reporting.

For stage planning, add a horizontal timeline at the bottom of the whiteboard with Phase 1 through Phase 7 so you can connect each branch to the delivery stage where it becomes active.

20. Final Build Summary

RIFCO should be built as a simple request app on the surface and a sophisticated pricing and dispatch intelligence system underneath. The MVP should launch with a narrow wedge, Oyster Shuckers, because that focus supports clearer marketing, tighter operations, and cleaner data collection.

The locked build direction is React Native with TypeScript for the mobile product, Firebase for the backend and real time logic foundation, native Swift and Kotlin modules for live tracking and background location support, and a Next.js operations layer on Vercel. This combination supports simultaneous iOS and Android launch while preserving product speed and technical quality.

The algorithm strategy should begin with structured event collection, formula based pricing, zone scoring, and provider ranking logic. The team should resist the temptation to force a black box AI narrative too early. Real intelligence comes from high quality data capture, disciplined rules, consistent monitoring, and gradual optimization.

If executed with discipline, the MVP does more than launch an app. It creates the operating system for a broader request based experience company. The Oyster Shucker launch is the first proving ground for the platform, not the endpoint of the vision.